

2026-05-24 - GPIO and Interrupts

Goal

Learn how to configure GPIO pins in Zephyr for input and output, and how to respond to pin state changes using hardware interrupts.

What you will learn

- How to get a GPIO device using the Device Tree
- How to configure a pin as output (LED) and input (button)
- How to register a GPIO interrupt callback
- How to debounce a button in software

Overview

Zephyr's GPIO API abstracts hardware pin control across all supported boards. Pin configuration is done via the Device Tree, and the C API calls reference DT node labels.

Key concepts

Controlling an LED:

```
const struct gpio_dt_spec led = GPIO_DT_SPEC_GET(DT_ALIAS(led0), gpios);
gpio_pin_configure_dt(&led, GPIO_OUTPUT_ACTIVE);
gpio_pin_set_dt(&led, 1);    // on
gpio_pin_set_dt(&led, 0);    // off
gpio_pin_toggle_dt(&led);    // toggle
```

Reading a button:

```
const struct gpio_dt_spec btn = GPIO_DT_SPEC_GET(DT_ALIAS(sw0), gpios);
gpio_pin_configure_dt(&btn, GPIO_INPUT);
int val = gpio_pin_get_dt(&btn);
```

Interrupt callback:

```
static struct gpio_callback btn_cb;

void btn_handler(const struct device *dev, struct gpio_callback *cb, uint32_t masks)
{
    printk("Button pressed!\n");
}
```

```
gpio_pin_interrupt_configure_dt(&btn, GPIO_INT_EDGE_TO_ACTIVE);  
gpio_init_callback(&btn_cb, btn_handler, BIT(btn.pin));  
gpio_add_callback(btn.port, &btn_cb);
```

nRF52840 DK aliases

- led0 through led3 — onboard LEDs
- sw0 through sw3 — onboard buttons

Why this matters

GPIO is the foundation of hardware interaction. LEDs, buttons, relays, and many sensors are wired directly to GPIO pins. Interrupts let you respond instantly without polling.

Practice tasks

1. Toggle led0 every 500 ms using `gpio_pin_toggle_dt`.
2. Read sw0 in a loop and mirror its state to led1.
3. Use an interrupt on sw0 to toggle led0 on each press.

Example folder

See `src/2026-05-24_GPIO_and_Interrupts` for the day's code example.

Next topic

Tomorrow we learn how to define a custom PCB as a Zephyr board target.